



Homework Assignment

Week 8: List Mutability and Memory

Total Points: 100 — Due: Next Class



Name: _____

Date: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see `instructions.pdf`
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding Mutability (15 points)

Time estimate: 15 minutes

1.1 Exercise 1.1: Mutable vs Immutable Table (8 points)

Fill in the table to show your understanding of mutability concepts:

Data Type	Example	Mutable?	What happens when modified?
String	"hello"		
List	[1, 2, 3]		
Integer	42		
List	["a", "b"]		

1.2 Exercise 1.2: Quick Concept Check (7 points)

Circle the correct answer for each question:

1. When you modify a list element with `my_list[0] = 5`, does Python create a new list?
 - A. Yes, it always creates a new list
 - B. No, it modifies the existing list in place
 - C. Sometimes, depending on the list size

2. What does the `id()` function tell you?

- A. The length of the list
- B. The memory address of the object
- C. The index of an element

3. If two variables have the same `id()`, what does that mean?

- A. They have the same value
- B. They point to the same object in memory
- C. They are both lists

2 Part 2: Memory Diagrams and Code Tracing (25 points)

Time estimate: 30 minutes

2.1 Exercise 2.1: Draw Memory Diagrams (12 points)

For this code, draw memory diagrams showing what happens at each step:

```

1  # Step 1
2  scores = [85, 90, 78]
3
4  # Step 2
5  backup = scores
6
7  # Step 3
8  backup.append(95)
9
10 # Step 4
11 print(scores)
12 print(backup)

```

Your memory diagram after Step 2:

Draw two variable boxes (`scores` and `backup`) and show what they point to in memory. Use arrows to show references.

After Step 3, what happened to the list in memory?

What will Step 4 print?

`scores:` _____

`backup:` _____

2.2 Exercise 2.2: Trace the is Operator (8 points)

What will this program print? Trace through each line:

```

1 list1 = [1, 2, 3]
2 list2 = list1
3 list3 = [1, 2, 3]
4
5 print(list1 is list2)
6 print(list1 is list3)
7 print(list1 == list2)
8 print(list1 == list3)

```

Your predictions:

Line 1 output: _____ Why? _____

Line 2 output: _____ Why? _____

Line 3 output: _____ Why? _____

Line 4 output: _____ Why? _____

2.3 Exercise 2.3: Predict Aliasing Effects (5 points)

What will this code print? Think carefully about aliasing:

```

1 fruits = ["apple", "banana"]
2 more_fruits = fruits
3 more_fruits[0] = "mango"
4 print(fruits)

```

Output: _____

Explain why: _____

3 Part 3: Multiple Choice - Identity vs Equality (12 points)

Time estimate: 15 minutes

Circle the correct answer:

1. What is the difference between `is` and `==`?

- A. `is` checks values, `==` checks memory location
- B. `is` checks memory location, `==` checks values
- C. They are exactly the same
- D. `is` only works with numbers

2. When does `list2 = list1` create a copy?

- A. Always
- B. Never - it creates a reference
- C. Only for small lists

- D. Only if list1 is empty

3. Which creates an independent copy of a list?

- A. `copy = original`
- B. `copy = original.copy()`
- C. `copy = original.append()`
- D. `copy = id(original)`

4. What happens when you pass a list to a function?

- A. Python creates a copy of the list
- B. Python passes a reference to the original list
- C. The list cannot be modified in the function
- D. The function gets a new empty list

5. After this code, what is true?

```
1 a = [1, 2]
2 b = a
3 b.append(3)
```

- A. a is [1, 2] and b is [1, 2, 3]
- B. Both a and b are [1, 2, 3]
- C. a is unchanged but b is [1, 2, 3]
- D. Python will show an error

6. Which method does NOT create an independent copy?

- A. `new_list = old_list[:]`
- B. `new_list = list(old_list)`
- C. `new_list = old_list.copy()`
- D. `new_list = old_list`

4 Part 4: Code Completion - Copying Methods (15 points)

Time estimate: 20 minutes

4.1 Exercise 4.1: Safe List Operations (8 points)

Open [week08_ex4.1.py](#) and complete this code that safely works with lists:

```

1  # Safe List Manager
2  print("=== Original and Backup Lists ===")
3
4  # Original shopping list
5  original_list = ["milk", "bread", "eggs"]
6  print(f"Original: {original_list}")
7
8  # Create a TRUE COPY (not a reference!)
9  backup_list = original_list.-----
10
11 # Modify the original
12 original_list.append("butter")
13 original_list.remove("bread")
14
15 # Show both lists
16 print(f"Modified original: {original_list}")
17 print(f"Backup (should be unchanged): {backup_list}")
18
19 # Verify they are different objects
20 print(f"Are they the same object? {original_list ___ backup_list}")
21 print(f"Do they have same values? {original_list ___ backup_list}")

```

4.2 Exercise 4.2: Three Copying Methods (7 points)

Open [week08_ex4.2.py](#) and complete this comparison of copying methods:

```

1  # Comparing Copy Methods
2  numbers = [10, 20, 30, 40]
3
4  # Method 1: .copy()
5  copy1 = numbers.-----
6
7  # Method 2: list() constructor
8  copy2 = -----(numbers)
9
10 # Method 3: slice notation
11 copy3 = numbers-----
12
13 # Verify all are independent
14 print(f"Original: {numbers}")
15 print(f"Copy 1: {copy1}")
16 print(f"Copy 2: {copy2}")
17 print(f"Copy 3: {copy3}")
18
19 # Modify original
20 numbers.append(50)
21
22 # Verify copies unchanged
23 print(f"\nAfter modifying original:")
24 print(f"Original: {numbers}")
25 print(f"Copy 1: {copy1}")
26 print(f"Copy 2: {copy2}")
27 print(f"Copy 3: {copy3}")

```

5 Part 5: Debugging Aliasing Bugs (18 points)

Time estimate: 25 minutes

Each code snippet has aliasing problems. Find and fix ALL errors:

5.1 Snippet 1: Unexpected Modification (9 points)

```

1  # This code has an aliasing bug!
2  # The backup gets modified when we don't want it to
3
4  class_grades = [85, 90, 78, 92]
5  backup_grades = class_grades # Problem here!
6
7  # Add bonus points to original
8  for i in range(len(class_grades)):
9      class_grades[i] = class_grades[i] + 5
10
11 print("Original with bonus:", class_grades)
12 print("Backup (should be old grades):", backup_grades)
13 # BUG: backup also changed!

```

What's wrong? _____

Fixed version:

```

1  # Write the corrected code here:

```

Save your corrected code as `week08_ex5-1_fixed.py`

5.2 Snippet 2: Function Aliasing Problem (9 points)

```

1  # This function has an aliasing bug!
2  def add_bonus_points(score_list):
3      # We want to add 10 to each score
4      # WITHOUT modifying the original
5      modified = score_list # Problem here!
6
7      for i in range(len(modified)):
8          modified[i] = modified[i] + 10
9
10     return modified
11
12 test_scores = [70, 80, 90]
13 new_scores = add_bonus_points(test_scores)
14
15 print("Original scores:", test_scores)
16 print("With bonus:", new_scores)
17 # BUG: Original also changed!

```

What's wrong? _____

Fixed version:

```

1  # Write the corrected code here:

```

Save your corrected code as `week08_ex5_2_fixed.py`

6 Part 6: Full Program Development (20 points)

Time estimate: 35 minutes

6.1 Exercise 6.1: Grade Modifier with Backup (20 points)

Create a program called `grade_modifier.py` that manages student grades with proper backup handling:

Program Requirements

Your program should:

- Start with a list of at least 5 student grades
- Create a TRUE COPY as backup
- Show a menu with options:
 - Add bonus points to all grades
 - Remove lowest grade
 - Show current grades
 - Restore from backup
 - Exit
- IMPORTANT: When modifying grades, the backup should remain unchanged
- Allow user to restore original grades from backup
- Use proper list copying methods
- Display memory confirmation (use `is` to verify backup is separate)

Example Run:

```
=== Grade Modifier System ===
Original grades: [75, 82, 68, 91, 88]
Backup created successfully!
Backup is independent: True

Menu:
1. Add 5 bonus points to all grades
2. Remove lowest grade
3. Show current grades
4. Restore from backup
5. Exit
Choice: 1

Adding 5 points to all grades...
Current grades: [80, 87, 73, 96, 93]
Backup (unchanged): [75, 82, 68, 91, 88]
```

Menu:

1. Add 5 bonus points to all grades
2. Remove lowest grade
3. Show current grades
4. Restore from backup
5. Exit

Choice: 4

Restoring from backup...

Grades restored: [75, 82, 68, 91, 88]

Starter Structure:

```

1  # grade_modifier.py
2  # Name: -----
3  # Date: -----
4
5  print("=== Grade Modifier System ===")
6
7  # Original grades
8  grades = [75, 82, 68, 91, 88]
9  print(f"Original grades: {grades}")
10
11 # Create TRUE COPY for backup
12 backup = -----
13
14 # Verify backup is independent
15 print(f"Backup created successfully!")
16 print(f"Backup is independent: {grades is not backup}")
17
18 # Main program loop
19 while True:
20     print("\nMenu:")
21     print("1. Add 5 bonus points to all grades")
22     print("2. Remove lowest grade")
23     print("3. Show current grades")
24     print("4. Restore from backup")
25     print("5. Exit")
26
27     choice = input("Choice: ")
28
29     if choice == "1":
30         # Add bonus points
31         # Your code here
32         pass
33
34     elif choice == "2":
35         # Remove minimum grade
36         # Your code here
37         pass
38
39     elif choice == "3":
40         # Show grades
41         # Your code here
42         pass
43
44     elif choice == "4":
45         # Restore from backup
46         # IMPORTANT: Create new copy of backup!
47         # Your code here

```



```

48     pass
49
50     elif choice == "5":
51         print("Goodbye!")
52         break
53     else:
54         print("Invalid choice!")

```

Testing Checklist:

- ☐ Program runs without errors
- ☐ Backup is created as independent copy (not reference)
- ☐ Modifying grades does NOT change backup
- ☐ Restore function works correctly
- ☐ Menu loop continues until user exits
- ☐ All options work as expected
- ☐ Output clearly shows backup is unchanged

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. Why is understanding mutability important when working with lists?

2. Explain the difference between `is` and `==` in your own words, using an example.

3. When would you WANT aliasing (sharing a reference), and when would you want a true copy?

8 Bonus Section: Extra Challenges (Optional - 10 extra points)

Only attempt if you've finished everything else!

8.1 Challenge 1: Nested List Aliasing (5 points)

Open [week08_bonus1.py](#) and explore nested list copying:

```

1  # Nested List Challenge
2  # What happens with nested lists?
3
4  matrix = [[1, 2], [3, 4]]
5  shallow_copy = matrix.copy()
6
7  # Modify inner list
8  shallow_copy[0][0] = 99
9
10 print(f"Original matrix: {matrix}")
11 print(f"Shallow copy: {shallow_copy}")
12
13 # Question: Did the original change? Why?
14 # Research: How would you make a "deep copy"?
15
16 # Your exploration here:

```

8.2 Challenge 2: Aliasing Detector Function (5 points)

Create a program called [aliasing_detector.py](#) that:

- Takes two list variables as input
- Checks if they are aliases (same object)
- Checks if they have equal values
- Creates a detailed report showing:
 - Memory addresses (using `id()`)
 - Whether they're the same object
 - Whether they have equal values
 - A conclusion about their relationship

Example output:

```

=== Aliasing Detector ===
List A ID: 140234567891000
List B ID: 140234567891000
Same object (aliased): True
Equal values: True
CONCLUSION: These are ALIASES - they point to the same list!

```

9 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

🔧 My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Saved each code exercise in its own file:
 - `week08_ex4_1.py`, `week08_ex4_2.py`
 - `week08_ex5_1_fixed.py`, `week08_ex5_2_fixed.py`
 - `grade_modifier.py`
- ☐ Created a folder named `week08_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Drawn memory diagrams where required
- ☐ Verified your understanding of aliasing vs copying
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Concepts Check:

Which concept was most challenging? _____

Do you understand the difference between `is` and `==`? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job mastering list mutability and memory!
Next week: List algorithms - searching and sorting techniques!